

Who, What, When, Where, Why

Look at the photo in figure 6 and tell me what you see. A woman petting a dog. A *soldier* petting a dog. A soldier who has just returned from war being welcomed by her dog, with flowers and a “Welcome Home” balloon. The soldier’s face shows her complex emotions. The dog is happily wagging its tail.

When was this photo taken? Most likely within the past ten years. Where does this photo take place? Probably an airport. Why is the soldier petting the dog? She has probably been away for a long time, experienced many things, both good and bad, missed her dog a great deal, and is very happy to be home. Perhaps the dog is a symbol of all that is “home.” What happened just before this photo was taken? The soldier probably got off an airplane and walked through the secure part of the airport to the place where passengers can be greeted. Her family or friends greeted her with hugs, handed her the flowers and balloon, and let go of the dog’s leash. The dog came over to the soldier, who put down everything she was carrying and knelt down, carefully putting the balloon’s string under her knee to keep it from floating off. What will happen next? She’ll probably stand up, maybe wipe away some tears, gather her flowers, balloon, and laptop computer, take the dog’s leash, and walk with the dog and her family or friends to the baggage claim area.



FIGURE 6: What do you see in this photo?

When you look at this picture, at the most basic level you're seeing bits of ink on a page (or pixels on a screen). Somehow your eyes and brain are able to take in this raw information and, within a few seconds, transform it into a detailed story involving living things, objects, relationships, places, emotions, motivations, and past and future actions. We look, we see, we understand. Crucially, we know what to ignore. There are many aspects of the photo that aren't strictly relevant to the story we extract from it: the pattern on the carpet, the hanging straps on the soldier's backpack, the whistle clipped to her pack's shoulder pad, the barrettes in her hair.

We humans perform this vast amount of information processing in hardly any time at all, and we have very little, if any, conscious awareness of what we're doing or how we do it. Unless you've been blind since birth, visual processing, at various levels of abstraction, dominates your brain.

Surely, the ability to describe the contents of a photograph (or a video, or a real-time stream from a camera) in this way would be one of the first things we would require for general human-level AI.

Easy Things Are Hard (Especially in Vision)

Since the 1950s, AI researchers have been trying to get computers to make sense of visual data. In the early days of AI, achieving this goal seemed relatively straightforward. In 1966, Marvin Minsky and Seymour Papert—the symbolic-AI-promoting MIT professors whom you’ll recall from chapter 1—proposed the Summer Vision Project, in which they would assign undergraduates to work on “the construction of a significant part of a visual system.”¹ In the words of one AI historian, “Minsky hired a first-year undergraduate and assigned him a problem to solve over the summer: connect a television camera to a computer and get the machine to describe what it sees.”²

The undergraduate didn’t get very far. And while the subfield of AI called computer vision has progressed substantially over the many decades since this summer project, a program that can look at and describe photographs in the way humans do still seems far out of reach. Vision—both looking and *seeing*—turns out to be one of the hardest of all “easy” things.

One prerequisite to describing visual input is *object recognition*—that is, recognizing a particular group of pixels in an image as a particular object category, such as “woman,” “dog,” “balloon,” or “laptop computer.” Object recognition is typically so immediate and effortless for us as humans that it didn’t seem as though it would be a particularly hard problem for computers, until AI researchers actually tried to get computers to do it.

What’s so hard about object recognition? Well, consider the problem of getting a computer program to recognize dogs in photographs. [Figure 7](#) illustrates some of the difficulties. If the input is simply the pixels of the image, then the program first has to figure out which are “dog” pixels and which are “non-dog” pixels (for example, background, shadows, other objects). Moreover, different dogs look very different: they can have diverse coloring, shape, and size; they can be facing in various directions; the lighting can vary considerably between images; parts of the dog can be blocked by other objects (for example, fences, people). What’s more, “dog pixels” might look a lot like “cat pixels” or other animals. Under some lighting conditions, a cloud in the sky might even look very much like a dog.



FIGURE 7: Object recognition: easy for humans, hard for computers

Since the 1950s, the field of computer vision has struggled with these and other issues. Until recently, a major job of computer-vision researchers was to develop specialized image-processing algorithms that would identify “invariant features” of objects that could be used to recognize these objects in spite of the difficulties I sketched above. But even with sophisticated image processing, the abilities of object-recognition programs remained far below those of humans.

The Deep-Learning Revolution

The ability of machines to recognize objects in images and videos underwent a quantum leap in the 2010s due to advances in the area called deep learning.

Deep learning simply refers to methods for training “deep neural networks,” which in turn refers to neural networks with more than one hidden layer. Recall that *hidden* layers are those layers of a neural network between the input and the output. The *depth* of a network is its number of hidden layers: a “shallow” network—like the one we saw in chapter 2—has only one hidden layer; a “deep” network has more than one hidden layer. It’s worth emphasizing this definition: the *deep* in deep learning doesn’t refer to the sophistication of what is learned; it refers only to the *depth in layers* of the network being trained.

Research on deep neural networks has been going on for several decades. What makes these networks a revolution is their recent phenomenal success in many AI tasks. Interestingly, researchers have found that the most successful deep networks are those whose structure mimics parts of the brain’s visual system. The “traditional” multilayer neural networks I described in chapter 2 were inspired by the brain, but their

structure is very un-brain-like. In contrast, the neural networks dominating deep learning are directly modeled after discoveries in neuroscience.

The Brain, the Neocognitron, and Convolutional Neural Networks

About the same time that Minsky and Papert were proposing their Summer Vision Project, two neuroscientists were in the midst of a decades-long study that would radically remake our understanding of vision—and particularly object recognition—in the brain. David Hubel and Torsten Wiesel were later awarded a Nobel Prize for their discoveries of hierarchical organization in the visual systems of cats and primates (including humans) and for their explanation of how the visual system transforms light striking the retina into information about what is in the scene.

Hubel and Wiesel's discoveries inspired a Japanese engineer named Kunihiko Fukushima, who in the 1970s developed one of the earliest deep neural networks, dubbed the cognitron, and its successor, the neocognitron. In his papers,³ Fukushima reported some success training the neocognitron to recognize handwritten digits (like the ones I showed in chapter 1), but the specific learning methods he used did not seem to extend to more complex visual tasks. Nonetheless, the neocognitron was an important inspiration for later approaches to deep neural networks, including today's most influential and widely used approach: convolutional neural networks, or (as most people in the field call them) ConvNets.

ConvNets are *the* driving force behind today's deep-learning revolution in computer vision, and in other areas as well. Although they have been widely heralded as the next big thing in AI, ConvNets are actually not very new: they were first proposed in the 1980s by the French computer scientist Yann LeCun, who had been inspired by Fukushima's neocognitron.

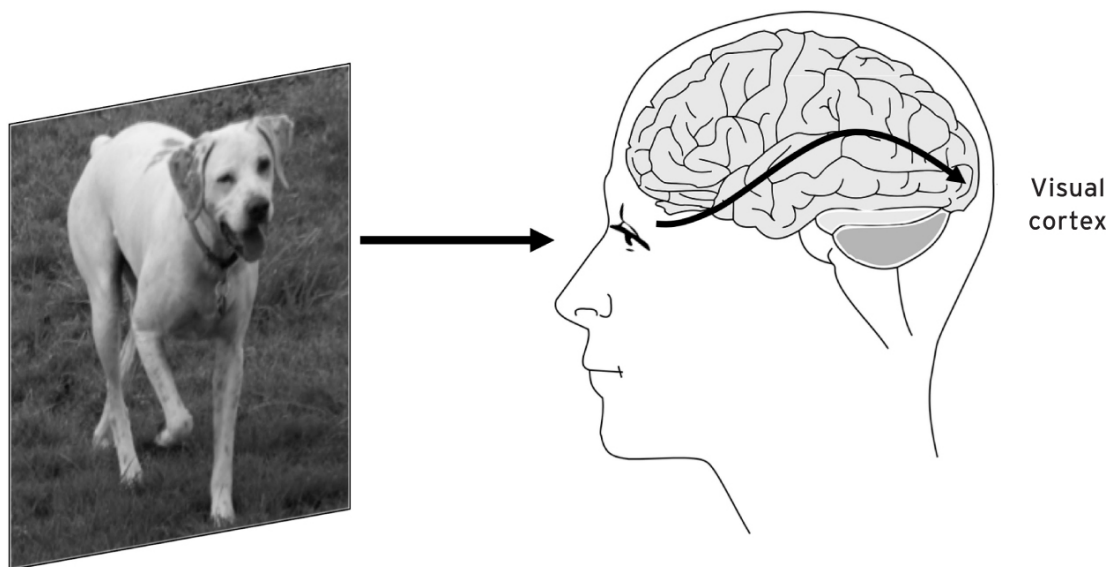


FIGURE 8: Pathway of visual input from eyes to visual cortex

I'll spend some time here describing how ConvNets work, because understanding them is crucial for making sense of where computer vision—as well as much else about AI—is today and what its limits are.

Object Recognition in the Brain and in ConvNets

Like the neocognitron, the design of ConvNets is based on several key insights about the brain's visual system that were discovered by Hubel and Wiesel in the 1950s and '60s. When your eyes focus on a scene, what they receive is light of different wavelengths that has been reflected by the objects and surfaces in the scene. Light falling on the eyes activates cells in each retina—essentially a grid of neurons in the back of the eye. These neurons communicate their activation through the optic nerves and into the brain, eventually activating neurons in the visual cortex, which resides in the back of the head ([figure 8](#)). The visual cortex is roughly organized as a hierarchical series of *layers* of neurons, like the stacked layers of a wedding cake, where the neurons in each layer communicate their activations to neurons in the succeeding layer.

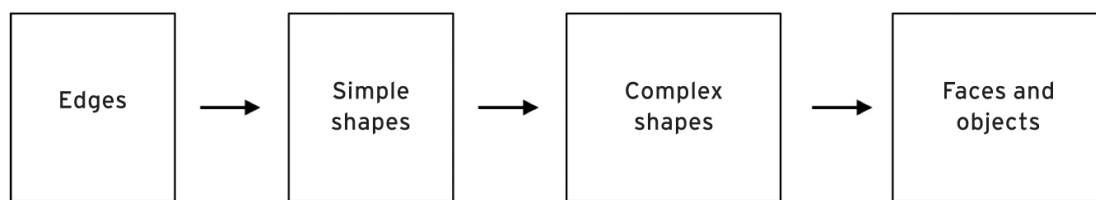


FIGURE 9: Sketch of visual features detected by neurons in different layers of the visual cortex

Hubel and Wiesel found evidence that neurons in different layers of this hierarchy act as “detectors” that respond to increasingly complex features appearing in the visual scene, as illustrated in [figure 9](#): neurons at initial layers become active (that is, fire at a higher rate) in response to edges; their activation feeds into layers of neurons that respond to simple shapes made up of these edges; and so on, up through more complex shapes and finally entire objects and specific faces. Note that the arrows in [figure 9](#) indicate a *bottom-up* (or *feed-forward*) flow of information, representing connections from lower to higher layers (in the figure, left to right). It’s important to note that a *top-down* (or *feed-backward*) flow of information (from higher to lower layers) also occurs in the visual cortex; in fact, there are about ten times as many feed-backward connections as feed-forward ones. However, the role of these backward connections is not well understood by neuroscientists, although it is well established that our prior knowledge and expectations, presumably stored in higher brain layers, strongly influence what we perceive.

Like the feed-forward hierarchical structure illustrated in [figure 9](#), a ConvNet consists of a sequence of layers of simulated neurons. I’ll again refer to these simulated neurons as *units*. Units in each layer provide input to units in the next layer. Just like the neural network I described in chapter 2, when a ConvNet processes an image, each unit takes on a particular *activation* value—a real number that is computed from the unit’s inputs and their weights.

Let’s make this discussion more specific by imagining a hypothetical ConvNet, with four layers plus a “classification module,” that we want to train to recognize dogs and cats in images. Assume for simplicity that each input image depicts exactly one dog or cat. [Figure 10](#) illustrates our ConvNet’s structure. It’s a bit complicated, so I’ll go through it carefully step-by-step to explain how it works.

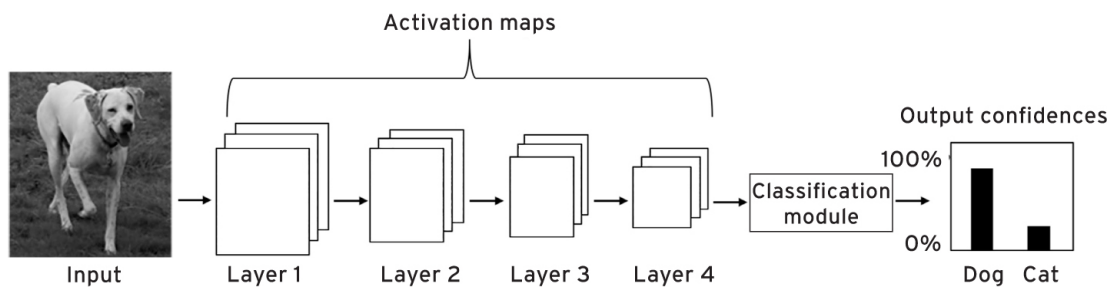


FIGURE 10: Illustration of a four-layer convolutional neural network (ConvNet) designed to recognize dogs and cats in photos

Input and Output

The input to our ConvNet is an image—that is, an array of numbers, corresponding to the brightness and color of the image’s pixels.⁴ Our ConvNet’s final output is the network’s confidence (0 percent to 100 percent) for each category: “dog” and “cat.” Our goal is to have the network learn to output a high confidence for the correct category and a low confidence for the other category. In doing so, the network will learn what set of *features* of the input image are most useful for this task.

Activation Maps

Notice in [figure 10](#) that each layer of the network is represented by a set of three overlapping rectangles. These rectangles represent *activation maps*, inspired by similar “maps” found in the brain’s visual system. Hubel and Wiesel discovered that neurons in the lower layers of the visual cortex are physically arranged so that they form a rough grid, with each neuron in the grid responding to a corresponding small area of the visual field. Imagine flying at night in an airplane over Los Angeles and taking a photo; the lights seen in your photo form a rough map of the features of the lit-up city. Analogously, the activations of the neurons in each grid-like layer of the visual cortex form a rough map of the important features in the visual scene. Now imagine that you had a very special camera that could produce separate photos for house lights, building lights, and car lights. This is something like what the visual cortex does: each important visual feature has its own separate neural map. The combination of these maps is a key part of what gives rise to our perception of a scene.

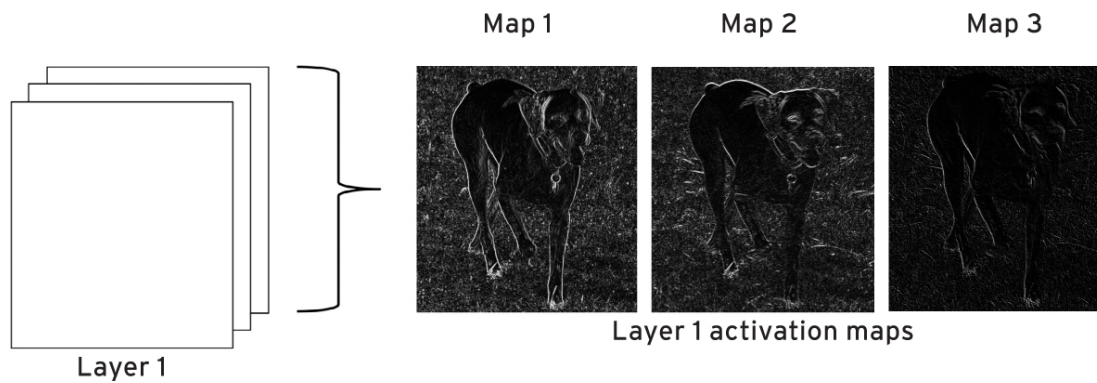


FIGURE 11: Activation maps in the first layer of our ConvNet

Like neurons in the visual cortex, the units in a ConvNet act as detectors for important visual features, each unit looking for its designated feature in a specific part of the visual field. And (very roughly) like the visual cortex, each layer in a ConvNet consists of several grids of these units, with each grid forming an activation map for a specific visual feature.

What visual features should ConvNet units detect? Let's look to the brain first. Hubel and Wiesel found that neurons in lower layers of the visual cortex act as edge detectors, where an *edge* refers to a boundary between two contrasting image regions. Each neuron receives input corresponding to a specific small region of the visual scene; this region is called the neuron's receptive field. The neuron becomes active (that is, starts firing more rapidly) only if its receptive field contains a particular kind of edge.

In fact, these neurons are quite specific about what kind of edge they respond to. Some neurons become active only when there is a vertical edge in their receptive field; some respond only to a horizontal edge; others fire only for edges at other specific angles. One of Hubel and Wiesel's most important findings was that each small region of your visual field corresponds to the receptive fields of many different such "edge detector" neurons. That is, at a low level of visual processing, your neurons are figuring out what edge orientations occur in every part of the scene you are looking at. Edge-detecting neurons feed into higher layers of the visual cortex, the neurons of which seem to be detectors for specific shapes, objects, and faces.⁵

Similarly, the first layer of our hypothetical ConvNet consists of edge-detecting units. [Figure 11](#) gives a closer view of layer 1 of our ConvNet.

This layer consists of three activation maps, each of which is a grid of units. Each unit in a map corresponds to the analogous location in the input image, and each unit gets its input from a small region around that location—its receptive field. (The receptive fields of neighboring units typically overlap.) Each unit in each map calculates an activation value that measures the degree to which the region “matches” the unit’s preferred edge orientation—for example, vertical, horizontal, or slanted at various degrees.

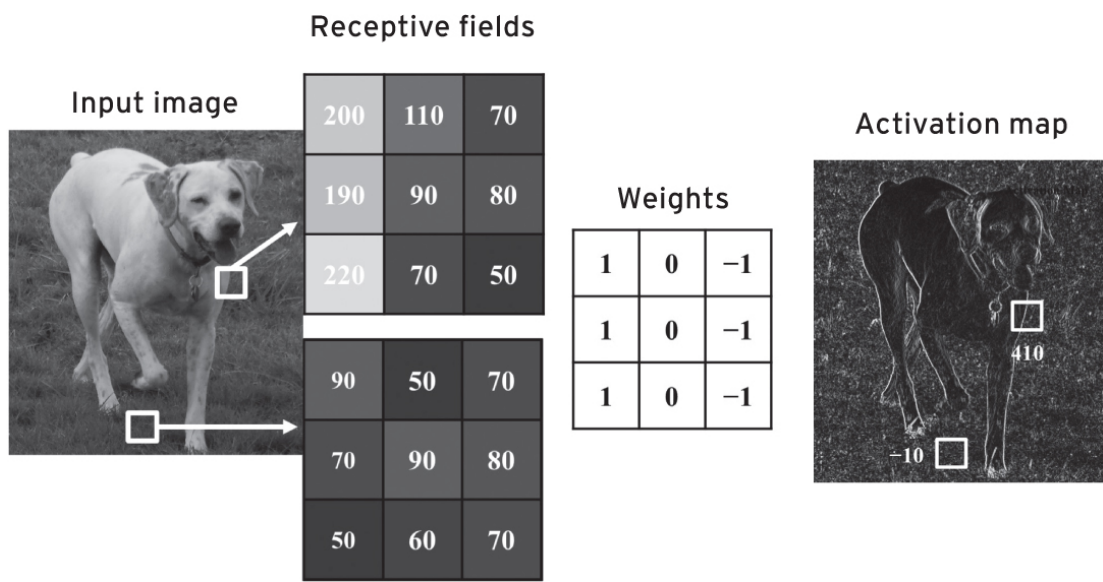


FIGURE 12: Illustration of how convolutions are used to detect vertical edges. For example, a convolution of the upper receptive field with the weights is $(200 \times 1) + (110 \times 0) + (70 \times -1) + (190 \times 1) + (90 \times 0) + (80 \times -1) + (220 \times 1) + (70 \times 0) + (50 \times -1) = 410$.

Figure 12 illustrates in detail how the units in map 1—those that detect vertical edges—calculate their activations. The small white squares in the input image represent the receptive fields of two different units. The image patches inside these receptive fields, when enlarged, are shown as arrays of pixel values. Here, for simplicity, I’ve displayed each patch as a three-by-three set of pixels (the values, by convention, range from 0 to 255—the lighter the pixel, the higher the value). Each unit receives as input the pixel values in its receptive field. The unit then multiplies each input by its weight and sums the results to produce the unit’s activation.

The weights shown in figure 12 are designed to produce a high positive activation when there is a light-to-dark vertical edge in the receptive field (that is, high contrast between the left and the right sides of the input patch).

The upper receptive field contains a vertical edge: the dog's light fur next to the darker grass. This is reflected in the high activation value (410). The lower receptive field does not contain such an edge, only dark grass, and the activation (−10) is closer to 0. Note that a dark-to-light vertical edge will yield a “high” negative value (that is, a negative value far from 0).

This calculation—multiplying each value in a receptive field by its corresponding weight and summing the results—is called a convolution. Hence the name “convolutional neural network.” I mentioned above that in a ConvNet, an activation map is a grid of units corresponding to receptive fields all over the image. Each unit in a given activation map uses the same weights to compute a convolution with its receptive field; imagine the input image with the white square sliding along every patch of the image.⁶ The result is the activation map in [figure 12](#): the center pixel of a unit's receptive field is colored white for high positive and negative activations and darker for activations close to 0. You can see that the white areas highlight the locations where vertical edges exist. Maps 2 and 3 in [figure 11](#) were created in the same way, but with weights that highlight horizontal and slanted edges, respectively. Taken together, the maps of edge-detecting units in layer 1 provide the ConvNet with a representation of the input image in terms of oriented edges in different regions, something like what an edge-detection program would produce.

Let's take a moment to talk about the word *map* here. In everyday use, *map* refers to a spatial representation of a geographic area, such as a city. A road map of Paris, say, shows a particular *feature* of the city—its layout of streets, avenues, and alleys—but doesn't include the city's many other features, such as buildings, houses, lampposts, trash cans, apple trees, and fishponds. Other kinds of maps focus on other features; you can find maps that highlight Paris's bike lanes, its vegetarian restaurants, its dog-friendly parks. Whatever your interests, there is quite possibly a map that shows where to find them. If you wanted to explain Paris to a friend who had never been there, a creative approach might be to show your friend a collection of such “special interest” maps of the city.

A ConvNet (like the brain) represents the visual scene as a collection of maps, reflecting the specific “interests” of a set of detectors. In my example in [figure 11](#), these interests are different edge orientations. However, as we'll see below, in ConvNets the network itself *learns* what its interests

(that is, detectors) should be; these depend on the specific task it is trained for.

Making maps isn't limited to layer 1 of our ConvNet. As you can see in [figure 10](#), a similar structure applies at all of the layers: each layer has a set of detectors, each of which creates its own activation map. A key to the ConvNet's success is that—again, inspired by the brain—these maps are *hierarchical*: the inputs to the units at layer 2 are the activation maps of layer 1, the inputs to the units at layer 3 are the activation maps of layer 2, and so on up the layers. In our hypothetical network, in which layer 1 units respond to edges, the layer 2 units would be sensitive to specific combinations of edges, such as corners and T shapes. Layer 3 detectors would be sensitive to combinations of combinations of edges. As you go up the hierarchy, the detectors become sensitive to increasingly more complex features, just as Hubel, Wiesel, and others saw in the brain.

Our hypothetical ConvNet has four layers, each with three maps, but in the real world these networks can have many more layers—sometimes hundreds—each with different numbers of activation maps. Determining these and many other aspects of a ConvNet's structure is part of the art of getting these complex networks to work for a given task. In chapter 3, I described I. J. Good's vision of a future “intelligence explosion” in which machines themselves create increasingly intelligent machines. We're not there yet. For the time being, getting ConvNets to work well requires a lot of human ingenuity.

Classification in ConvNets

Layers 1 to 4 of our network are called convolutional layers because each performs convolutions on the preceding layer (and layer 1 performs convolutions on the input). Given an input image, each layer successively performs its calculations, and finally at layer 4 the network has produced a set of activation maps for relatively complex features. These might include eyes, leg shapes, tail shapes, or anything else that the network has learned is useful for classifying the objects it is trained on (here dogs and cats). At this point, it's time for the classification module to use these features to predict what object the image depicts.

The classification module is actually an entire traditional neural network, similar to the kind I described in chapter 2.⁷ The inputs to the classification module are the activation maps from the highest convolutional layer. The module's output is a set of percentage values, one for each possible category, rating the network's confidence that the input depicts an image of that category (here dog or cat).

Let me summarize this brief explanation of ConvNets: Inspired by Hubel and Wiesel's findings on the brain's visual cortex, a ConvNet takes an input image and transforms it—via convolutions—into a set of activation maps with increasingly complex features. The features at the highest convolutional layer are fed into a traditional neural network (which I've called the classification module), which outputs confidence percentages for the network's known object categories. The object category with the highest confidence is returned as the network's classification of the image.⁸

Would you like to experiment with a well-trained ConvNet? Simply take a photo of an object, and upload it to Google's "search by image" engine.⁹ Google will run a ConvNet on your image and, based on the resulting confidences (over thousands of possible object categories), will tell you its "best guess" for the image.

Training a ConvNet

Our hypothetical ConvNet consists of edge detectors at its first layer, but in real-world ConvNets edge detectors aren't built in. Instead, ConvNets learn from training examples what features should be detected at each layer, as well as how to set the weights in the classification module so as to produce a high confidence for the correct answer. And, just as in traditional neural networks, all the weights can be learned from data via the same back-propagation algorithm that I described in chapter 2.

More specifically, here is how you could train our ConvNet to identify a given image as a dog or cat. First, collect many example images of dogs and cats—this is your "training set." Also, create a file that gives a label for each image—that is, "dog" or "cat." (Or better, take a hint from computer-vision researchers: Hire a graduate student to do all this for you. If you are a graduate student, then recruit an undergrad. No one enjoys this labeling

chore!) Your training program initially sets all the weights in the network to random values. Then your program commences training: one by one, each image is given as the input to the network; the network performs its layer-by-layer calculations and finally outputs confidence percentages for “dog” and “cat.” For each image, your training program compares these output values to the “correct” values; for example, if the image is a dog, then “dog” confidence should be 100 percent and “cat” confidence should be 0 percent. Then the training program uses the back-propagation algorithm to change the weights throughout the network just a bit, so that the next time this image is seen, the confidences will be closer to the correct values.

Following this procedure—input the image, then calculate the error at the output, then change the weights—for every image in your training set is called one “epoch” of training. Training a ConvNet requires many epochs, during which the network processes each image over and over again. Initially, the network will be very bad at recognizing dogs and cats, but slowly, as it changes its weights over many epochs, it will get increasingly better at the task. Finally, at some point, the network “converges”; that is, the weights stop changing much from one epoch to the next, and the network is (in principle!) very good at recognizing dogs and cats in the images in the training set. But we won’t know if the network is actually good at this task in general until we see if it can apply what it has learned to identify images from outside its training set. What’s really interesting is that, even though ConvNets are not constrained by a programmer to learn to detect any particular feature, when trained on large sets of real-world photographs, they indeed seem to learn a hierarchy of detectors similar to what Hubel and Wiesel found in the brain’s visual system.

In the next chapter, I’ll recount the extraordinary ascent of ConvNets from relative obscurity to near-complete dominance in machine vision, a transformation made possible by a concurrent technological revolution: that of “big data.”